

Anthony Marrs
CSCI 460 – Introduction to Applied Machine Learning
May 4, 2026

Abstract

Swarm behavior is a good example of how simple, local interactions between individuals can lead to coordinated patterns at a much larger scale, even without any central control. In this study, machine learning techniques were used to classify whether a swarm is behaving in a coordinated way using a high-dimensional dataset with about 72,000 observations and more than 2,400 numerical features. These features describe how individuals interact, including things like position, movement, alignment, spacing, and relationships with nearby individuals. The problem was treated as a binary classification task: coordinated versus non-coordinated behavior. (Bonabeau, Dorigo, & Theraulaz, 1999)

Initial data exploration showed a few challenges, including moderate class imbalance, many correlated features, and large differences in feature scales. To handle this, the data was preprocessed using mean imputation for missing values and standardization to bring features onto a consistent scale. Principal Component Analysis (PCA) was then applied to reduce the dataset down to 50 components while keeping about 55% of the original variance, making the data easier to work with.

Six models were tested: Logistic Regression, K-Nearest Neighbors, Decision Tree, Support Vector Machine (SVM), Random Forest, and XGBoost. Hyperparameters were tuned using cross-validation, and performance was evaluated using accuracy, precision, recall, and F1-score. Due to the class imbalance, the F1 score was used as the main focus. (Fawcett, 2006)

Among all models, SVM performed the best, achieving an F1-score of 0.9008. Overall, the results show that combining dimensionality reduction with the right model and tuning approach can effectively handle complex, high-dimensional swarm data.

1. Introduction

Swarm behavior is a system where large groups of individuals interact locally while exhibiting coordinated behavior at the group level without any central control. You can see this kind of behavior in nature, such as in flocks of birds, schools of fish, insect swarms, and even microbial colonies. In each of these cases, individuals follow relatively simple rules based on what is happening around them; but when you look at the group, the result appears organized and adaptive. The idea of swarm behavior has helped when it comes to understanding decentralized coordination and adaptability, gaining increased interest throughout scientific fields like biology, physics, computer science, and engineering.

One of the key ideas behind swarm systems is that coordination happens through local interactions rather than centralized decision-making, with individuals adjusting their behavior based on nearby individuals, typically by aligning with them, avoiding collisions, and staying close to the group. These ideas are often described as alignment, separation, and cohesion. (Reynolds, 1987) These ideas form the foundation for many swarm models. Though these rules are simple, the interactions between individuals quickly become non-linear. This makes the overall larger system difficult to predict using traditional methods. Swarm systems are often described as emergent systems, where large scale behavior is not easily understood simply by assessing individuals.

In recent years, swarm behavior has become more relevant in artificial systems as well, especially in areas related to swarm intelligence. For example, swarms of drones can work together to map environments, robotic swarms can be used in search-and-rescue missions, and distributed sensor networks can collect data in dynamic environments. These systems rely on decentralized control, which makes them more flexible and less likely to fail if one part of the system breaks. Swarm-based ideas are also used in optimization algorithms. For instance, particle swarm optimization utilizes this algorithm. (Kennedy & Eberhart, 1995) These applications are able to model and classify swarm behavior, resulting in system designs that can adapt and scale effectively. (Brambilla et al., 2013)

The task of modeling swarm behavior is not straightforward. The interactions between individuals depend on multiple factors, including position, velocity, orientation, and environmental conditions. This leads to high-dimensional data, where each observation may include hundreds or thousands of features. Traditional approaches, such as differential equations or rule-based simulations, can capture some aspects of swarm behavior, but they often struggle when the system becomes too complex or large. In addition, these approaches usually depend on assumptions about how the system behaves, and those assumptions do not always hold in real-world scenarios.

Machine learning offers a different way to approach this problem by learning patterns directly from the data instead of relying on predefined rules. In supervised learning, models are trained to map input features to labels, which allows them to classify different swarm states based on observed patterns. In practice, this works especially well for high-dimensional datasets, where relationships between features can be difficult to model explicitly. Machine learning models can capture nonlinear interactions, identify which features matter most, and make predictions on new data, making them a useful tool for studying swarm systems.

The dataset used in this project comes from a simulated swarm environment and includes around 72,000 observations with more than 2,400 features. Each observation represents a snapshot of the system at a specific point in time and includes detailed information about how individuals interact. The features describe traits like position, velocity, alignment, separation,

cohesion, and relationships between neighboring individuals. The target variable is binary, meaning the model needs to determine whether coordinated swarm behavior is present or not.

One of the main challenges with this dataset is high dimensionality. With over 2,400 features, the dataset becomes more difficult to work with both computationally and from a modeling standpoint. As the total number of features increases, the feature space expands quickly, making it harder for models to learn meaningful patterns. Data points can also become more widespread, and distance-based measures begin to become less reliable. Another challenging factor is that many features can and are correlated with each other, meaning they contain overlapping information that does not necessarily improve performance.

Class imbalance is another factor that interferes with performance. If one class appears more often than the other, it can cause models to lean toward predicting the majority class. In this situation, accuracy alone is not a good measure of performance, since a model could achieve high accuracy without learning useful patterns. Metrics such as precision, recall, and F1-score provide a better picture because they take into account both types of classification errors.

To deal with high dimensionality and feature redundancy, dimensionality reduction techniques are needed. Principal Component Analysis (PCA) is one common method that reduces the number of features while keeping the most important information. PCA works by transforming the data into a new set of components that capture the directions of highest variance. This makes the dataset easier to work with and can help reduce overfitting. However, it also means the transformed features are harder to interpret, since they no longer correspond directly to the original variables. (Jolliffe, 2002)

The main goal of this project is to compare different machine learning models and determine which one performs best for this classification task. The paper assesses how preprocessing steps, dimensionality reduction, and hyperparameter tuning affect machine learning performance. By testing different types of models, including linear models, distance-based methods, tree-based approaches, and ensemble techniques, it becomes possible to see how different methods handle this type of data.

The six models used in this study include: Logistic Regression, K-Nearest Neighbors, Decision Tree, SVM, Random Forest, and XGBoost. Logistic Regression provides a simple baseline, while KNN focuses on local structure. Decision Trees capture nonlinear relationships, and SVM is known for handling high-dimensional data well. Random Forest and XGBoost combine multiple models to improve performance and reduce overfitting. Together, these models offer a broad comparison across different machine learning approaches.

Hyperparameter tuning also plays a key role in improving model performance. This is because models rely on parameters that control how they learn, such as the number of neighbors in KNN, or the depth of trees in Random Forest. In this project, tuning was done using

GridSearchCV and RandomizedSearchCV with cross-validation. Doing so allows for different parameter combinations to be tested in a systematic way.

As previously noted, utilizing robotics with swarm-based systems allows groups of simple robots to perform complex tasks like mapping environments or navigating difficult terrain. In distributed systems, similar ideas are used for routing, load balancing, and resource management. These systems benefit from decentralization, which makes them more flexible and less likely to fail. Swarm behavior is applied in biological and ecological research, while studying how individuals move and interact in groups can provide insight into how they respond to their environment, avoid predators, and use resources efficiently. Machine learning can help identify patterns in these systems that are not always obvious through observation alone, which can lead to better models and new ideas for bio-inspired algorithms.

Throughout this paper, machine learning models will be utilized to analyze exploratory data and predict patterns within the dataset. This is shown through data analysis, methodology on how the data was prepared, how the models were built, evaluated, and performance analysis. The results are discussed while acknowledging limitations of this paper and possible directions for future work.

2. Exploratory Data Analysis

Exploratory Data Analysis (EDA) is a fundamental step to building a machine learning model. This is because it helps to understand what variables will be utilized before jumping into training. In this project, EDA was especially important because the dataset is both high-dimensional and fairly complex. By assessing the data early on, it becomes easier to spot issues like class imbalance, overlapping features, and differences in how features are distributed. These observations ended up guiding many of the decisions made later during preprocessing and modeling.

The dataset used in this study contains about 72,000 observations, where each one represents a snapshot of a swarm system at a specific point in time. Each observation includes more than 2,400 numerical features, which describe different aspects of how individuals interact. These features include things like an individual's position, velocity, alignment, separation, cohesion, and relationships between neighboring individuals. With the number of features involved, the swarm behavior can be influenced by a large number of interacting variables. As well as the high dimensionality of data make the dataset harder to work with, both in terms of computation and model performance.

One of the first things examined during EDA was the distribution of the target variable. The dataset shows moderate class imbalance, with about 43,500 observations in Class 0 and around 28,500 in Class 1. Class 0 represents non-coordinated swarm behavior, while Class 1

represents coordinated behavior. The class distribution plot makes this imbalance easy to see, with Class 0 appearing more frequently. While the imbalance is not extreme, it is enough to affect how models behave. In simulations, models trained on imbalanced data tend to favor the majority class, which can lead to high accuracy but weaker performance on the minority class. It makes more sense to rely on evaluation metrics that consider both precision and recall. For this reason, F1-score was used as the main evaluation metric, since it provides a more balanced view of performance across both classes.

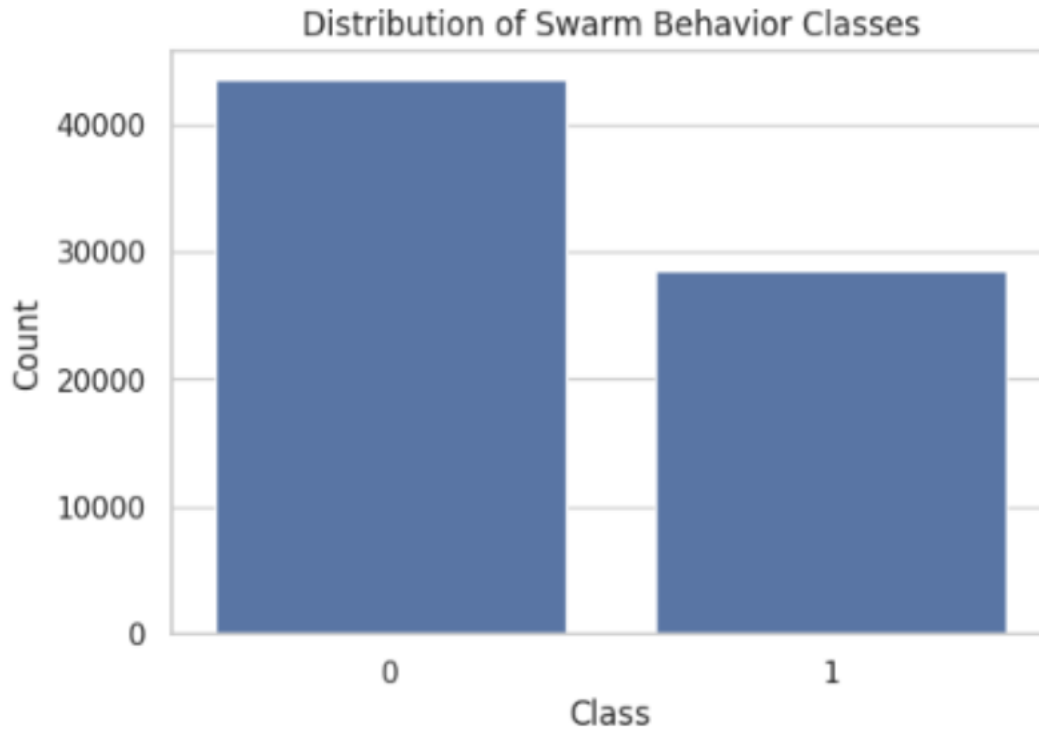


Figure 1 illustrates the moderate imbalance between coordinated and non-coordinated swarm behavior.

Beyond the target variable, it was also important to understand how the individual features behave. Histograms and summary statistics were generated for a subset of representative features, and a few patterns stood out. Many features show skewed distributions rather than following a normal shape, and their values are often spread across very different ranges. Some features stay within a narrow range, while others extend much farther and include long tails. From the histograms, it becomes clear that the dataset does not follow consistent distributions across features, which can affect how certain models perform.

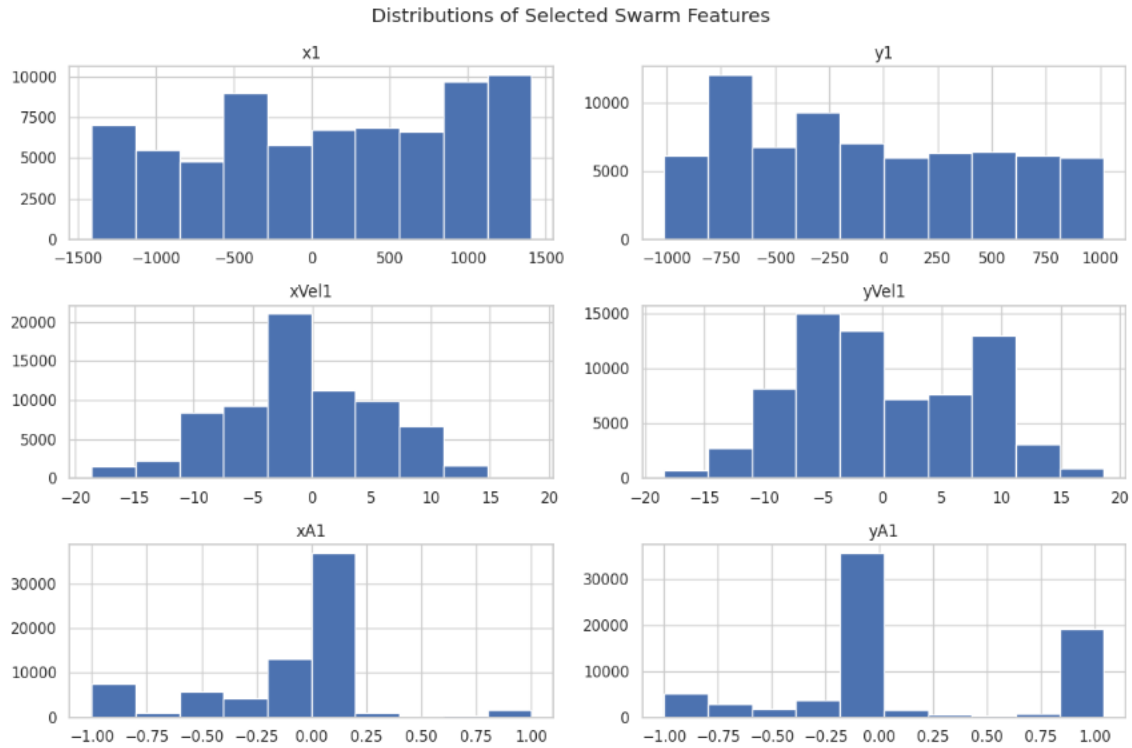


Figure 2 shows the variability across selected swarm features.

Because of these differences in scale, feature normalization becomes necessary. Without scaling, features with larger values would have a much stronger influence on the model compared to smaller ones. This is especially noticeable in models like K-Nearest Neighbors and SVMs, where distance calculations are used to compare data points. If the features are not scaled, those distance calculations can become distorted, which leads to weaker performance. Applying `StandardScaler` helps fix this by putting all features on a similar scale so they contribute more evenly during training.

Another important part of EDA was looking at how features relate to each other. A correlation matrix was created and visualized using a heatmap, which showed that many features are strongly correlated. This means that several features are capturing similar information about the system.

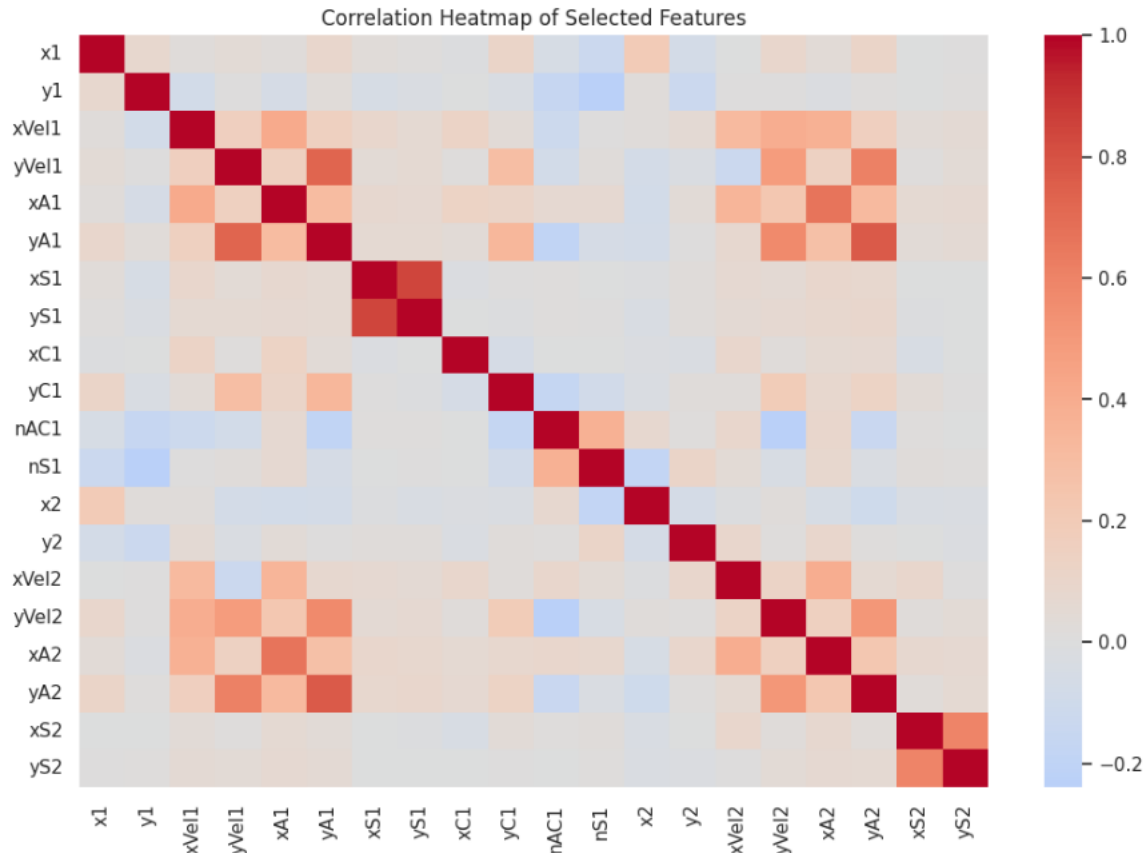


Figure 3 shows strong correlations among features, indicating redundancy.

This kind of redundancy is common in high-dimensional datasets, especially when the data comes from simulations. Having extra features does not necessarily hurt performance, though it can increase the complexity that they need to be and increase training time. It can also cause issues for models that are sensitive to correlated inputs. Reducing the number of features becomes an important step.

The presence of strong correlations directly points toward using dimensionality reduction techniques like PCA. When features are highly correlated, it is often possible to represent the same information using fewer variables. PCA works by transforming the original features into a smaller set of components that capture the most variation in the data. This helps simplify the dataset while still keeping the most important patterns, which makes it easier for models to learn effectively.

Another challenge that comes up with dimensionality is the curse of dimensionality. As the number of features increases, the feature space becomes much larger, and data points spread out more. It makes it harder for models to find meaningful patterns. Distance-based models like KNN are affected because distances between points become less informative in high-dimensional spaces. It also increases

the risk of overfitting, where the model learns noise rather than general patterns. Reducing dimensionality and choosing the right models becomes even more important.

Outliers were also considered during the analysis. Some features showed long tails, which suggests the presence of extreme values. Instead of removing these values, they were kept in the dataset. In a system like swarm behavior, unusual values may actually represent valid but less common patterns rather than noise. Removing them could lead to losing useful information. Scaling was used to reduce their impact during training.

In this particular analysis, there was a small number of missing values found after converting everything to numeric form. This means the dataset is relatively clean and does not require extensive data cleaning. The few missing values that did exist were handled using mean imputation, which works well for numerical data and has very little impact when the dataset is large.

Overall, the EDA phase helped identify the main characteristics of the dataset and how they would affect the modeling process. The class imbalance further showed why metrics like F1-score are needed. The variation in feature distributions made scaling necessary, and the presence of correlated features supported the use of PCA. At the same time, keeping outliers and handling minimal missing data helped preserve the structure of the dataset. These insights made it possible to build a more effective modeling approach.

The EDA also gives a sense of how different models might perform. For example, the strong correlations between features suggest that models assuming independent features may struggle. The high dimensionality also means that models sensitive to feature space size, like KNN, could perform worse unless dimensionality reduction is applied. This reinforces the idea that model selection should match the structure of the data rather than using a one-size-fits-all approach.

Another interesting observation is that not all features behave the same way. Some features, especially those related to velocity and alignment, tend to change more over time, while others are more stable. This difference may help explain how models distinguish between coordinated and non-coordinated behavior. Coordinated systems often show more consistent movement patterns, so features related to motion may play a larger role in classification. Identifying these patterns early on helps explain why certain models perform better later in the analysis.

3. Methodology

The methodology for this study addressed the challenges identified during the exploratory data analysis while making sure the models were trained and evaluated in a

consistent and reliable way. Because the dataset has high dimensionality, and is somewhat imbalanced, and complex; the overall approach followed a structured pipeline. This included preprocessing, dimensionality reduction, model selection, hyperparameter tuning, and evaluation. Each step was chosen based on what the data looked like and what the classification task required.

Initially, the dataset for modeling needed to be prepared. All features were converted into numerical format so they could be used by machine learning algorithms, since most models require numeric input. After that, the dataset was checked for missing values. During the exploratory analysis, it was found that there were very few missing entries. These were handled using mean imputation, where missing values are replaced with the average value of that feature. This approach was utilized because it keeps the overall distribution of the data intact. Given the size of the dataset, these small adjustments are unlikely to have much impact on overall performance.

Once missing values were resolved, the next step was feature scaling. StandardScaler was used to standardize the data so that each feature has a mean of zero and a standard deviation of one. This step becomes especially important for models like K-Nearest Neighbors and SVMs, which rely on distance calculations. Without scaling, features with larger values would have more influence on the model, which could lead to biased results. Scaling ensures that all features contribute more evenly, allowing the model to learn patterns more effectively.

Because the dataset contains more than 2,400 features, dimensionality reduction was necessary to make the problem more manageable. Principal Component Analysis (PCA) was used to reduce the number of features to 50 components. PCA works by transforming the original features into a new set of components that capture the most variation in the data. In this case, reducing the dataset to 50 components still retained about 55% of the total variance. This simplified the dataset while keeping the information. The choice of 50 components was based on the explained variance curve, which showed that adding more components beyond that point did not add much additional value.

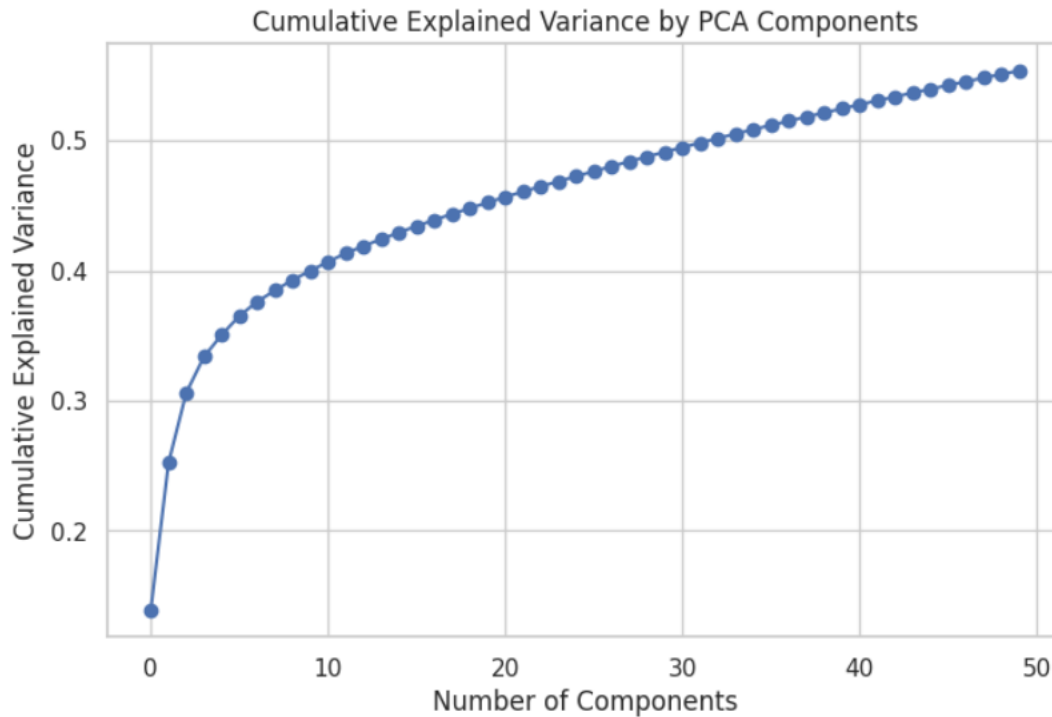


Figure 4 shows that around 50 components capture most of the meaningful variance in the dataset.

Using PCA comes with a tradeoff. While it makes the dataset easier to work with and helps reduce overfitting, it also makes the features harder to interpret. The new components do not directly correspond to the original features, which makes it more difficult to understand exactly what the model is using to make predictions. Even with this, PCA made sense for this dataset because of the high level of redundancy identified earlier. By reducing correlated features into a smaller set of components, the model is able to generalize better while keeping the feature space manageable.

After preprocessing and dimensionality reduction, several machine learning models were selected for evaluation. The goal was to include a variety of approaches so it would be possible to compare how different types of models perform on this kind of data. Logistic Regression was included as a baseline model because it is simple and easy to interpret. Since it assumes a linear relationship between features and the target variable, it provides a good reference point for evaluating more complex models.

K-Nearest Neighbors was included as a distance-based model that classifies data points based on nearby observations. This model works well when similar points are close together in the feature space. This becomes less effective when the number of features is extremely high. PCA helps improve its performance by reducing the dimensionality of the data, which makes distance calculations more meaningful.

Decision Trees were used because they can capture nonlinear relationships between features. They work by splitting the data into smaller groups based on feature values, which allows them to model complex interactions. Though one fall back is that they can also overfit the data, especially when the dataset is large and contains many features. This means they may perform well on training data but not as well on new data.

SVMs were selected because they tend to perform well in high-dimensional spaces. SVM works by finding a boundary that best separates the classes, focusing on the data points that are closest to that boundary. This is one reason SVM performs well on complex datasets, since it emphasizes the most important observations. Kernel functions can also be used to capture nonlinear relationships, which makes SVM a strong option for this type of problem.

Random Forest and XGBoost were included as ensemble methods. Random Forest builds multiple decision trees and combines their predictions, which helps reduce overfitting and improve generalization. XGBoost takes a different approach by building trees one at a time, where each new tree focuses on correcting the errors made by the previous ones. This step-by-step improvement process allows XGBoost to capture complex patterns in the data. Both models are commonly used because they tend to perform well on a wide range of problems.

Another factor considered during model selection was the balance between bias and variance. Simpler models like Logistic Regression tend to underfit but produce stable results, while more complex models like Decision Trees and ensemble methods can fit the data more closely but may overfit. By including multiple models, it makes it easier to compare how different approaches handle the complexity of the dataset.

Reproducibility was also an important part of the methodology. By using consistent preprocessing steps, clearly defined data splits, and cross-validation for tuning, the results can be reproduced and verified. This helps ensure that the findings are reliable and not dependent on random chance.

To evaluate the models, the dataset was split into training and testing sets using an 80/20 split. Stratified sampling was used so that both sets would maintain the same class distribution. The training set was used to train the models and tune parameters, while the testing set was used to evaluate performance on unseen data. Keeping these sets separate helps provide a more accurate measure of how well the models generalize.

Hyperparameter tuning was done using GridSearchCV and RandomizedSearchCV. GridSearchCV tests all combinations of selected parameter values, while RandomizedSearchCV samples a smaller set of combinations. This was to find parameter settings that improve model performance. For example, parameters like the number of neighbors in KNN, tree depth in Random Forest, and regularization in SVM were adjusted to get better results. Cross-validation

was used during this process to make sure the chosen parameters work well across different groups of data.

Finally, model performance was evaluated using multiple metrics, including accuracy, precision, recall, and F1-score. Accuracy measures overall correctness, but it does not account for class imbalance. Precision and recall provide more detail about how the model performs on positive predictions, and F1-score combines both into a single value. Because the dataset is imbalanced, F1-score was used as the main metric. Confusion matrices were also used to break down predictions into true positives, true negatives, false positives, and false negatives, which helps give a clearer picture of model performance.

Overall, the methodology used in this study was designed to handle the challenges of high-dimensional swarm data while producing reliable results. By combining preprocessing, dimensionality reduction, a variety of models, and careful tuning, the approach provides a clear framework for analyzing swarm behavior using machine learning. Each step was based on what the data showed, which helped ensure that the models were both effective and appropriate for the problem.

4. Results and Discussion

This study provides a clear comparison of how each model performed on the swarm behavior classification task. All models were evaluated using accuracy, precision, recall, and F1-score. In this case, F1-score is the most important metric because of the class imbalance in the dataset. The study also looked at how hyperparameter tuning affected the models and what might explain the differences in their results.

Looking at the overall performance, the SVM stands out as the top-performing model, achieving an F1-score of about 0.9008. XGBoost followed closest behind, with a F1-score of 0.885. Logistic Regression and K-Nearest Neighbors also perform well, with F1-scores around 0.87 to 0.88, showing that even simpler models are able to capture useful patterns in the data. The Decision Tree model, however, performs noticeably worse, with an F1-score of about 0.74, which suggests it struggles to generalize in this high-dimensional setting.

	Model	Accuracy	Precision	Recall	F1
3	SVM	0.917141	0.856015	0.950561	0.900814
5	XGBoost	0.905552	0.851888	0.921634	0.885389
0	Logistic Regression	0.902706	0.873827	0.881487	0.877640
1	KNN	0.895836	0.839225	0.911466	0.873855
4	Random Forest	0.860583	0.812659	0.841865	0.827004
2	Decision Tree	0.809993	0.800324	0.692847	0.742718

Table 1 summarizes the performance of all models across evaluation metrics.

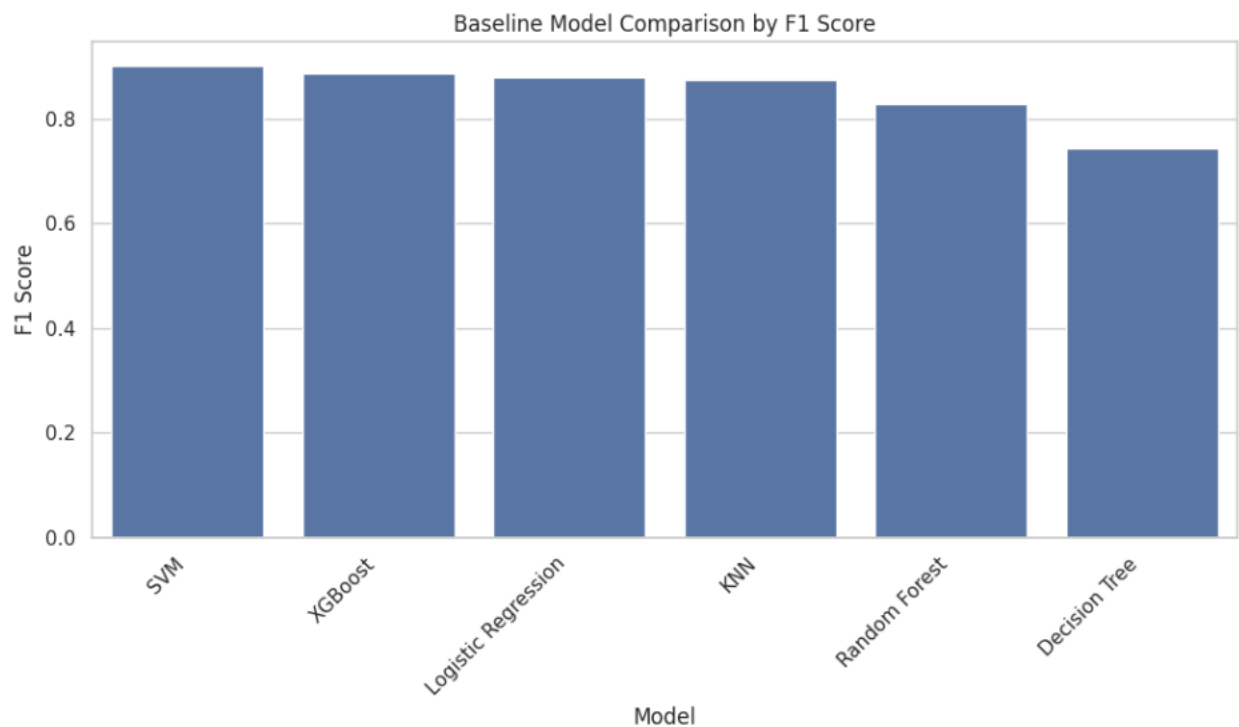


Figure 5 provides a visual comparison that shows how SVM slightly outperforms the others.

There are a few reasons why the SVM performed better than the other models. One reason is that SVM is well-suited for high-dimensional datasets. Instead of relying on all features equally, it focuses on finding the best boundary between classes by maximizing the margin. This allows it to perform well even when the number of features is large compared to the number of observations. Even after PCA reduced the dataset, the feature space was still complex enough for SVM to take advantage of this. Another factor is that SVM focuses on the most informative data points, known as support vectors, which helps reduce the influence of less useful observations and improves generalization.

SVM also has the advantage of handling nonlinear relationships through kernel functions. Since swarm behavior comes from complex interactions between individuals, the relationships in the data are not purely linear. While models like Logistic Regression may miss some of these patterns, SVM can transform the data into a space where it becomes easier to separate the classes. This is one of the main reasons it performs so well on this problem.

Hyperparameter tuning also played a role in improving model performance. Tuning was done using GridSearchCV and RandomizedSearchCV, which test different parameter combinations using cross-validation. Looking at the results, most models improved after tuning, although the amount of improvement varied. Ensemble models like Random Forest and XGBoost benefited the most, since their performance depends heavily on parameters such as tree depth, number of estimators, and learning rate. After tuning, these models were able to generalize better and achieve higher F1-scores.

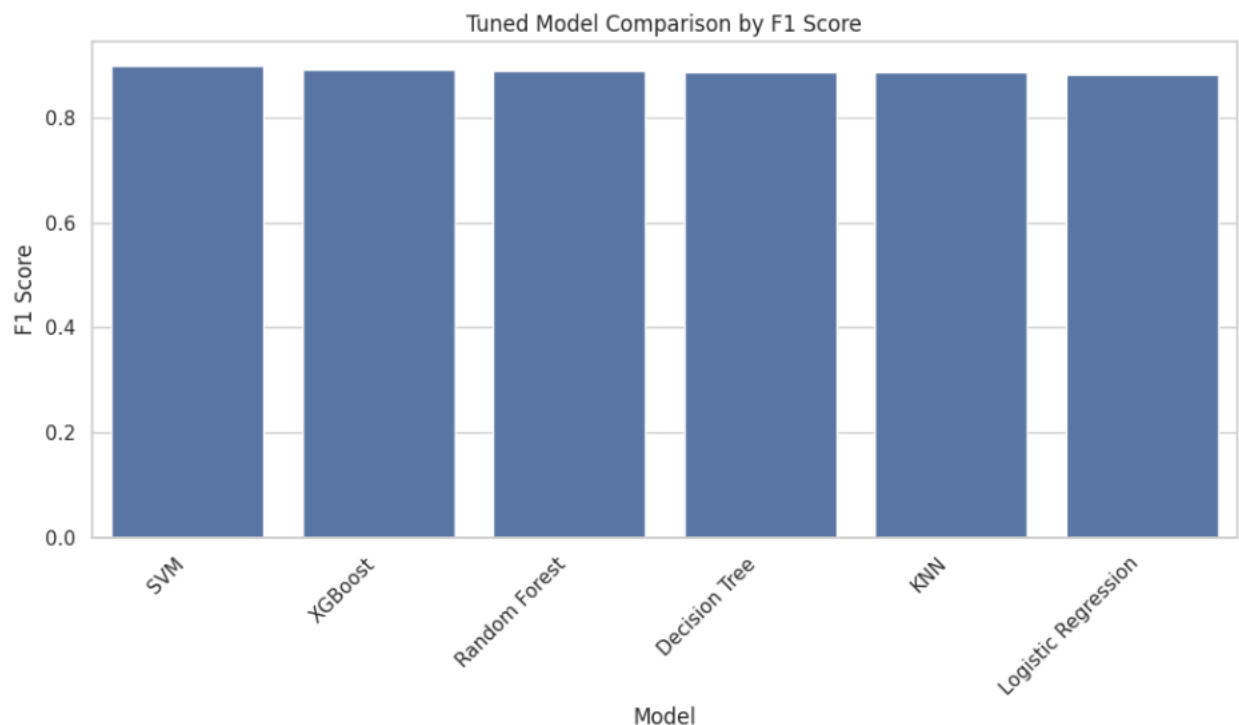


Figure 6 shows how model performance changed before and after hyperparameter tuning.

SVM, on the other hand, did not change much after tuning. For this analysis, the default parameters were already a good fit for the dataset. This also suggests that SVM is less sensitive to parameter changes compared to more complex models. Logistic Regression and KNN showed smaller improvements, which makes sense given their simpler structure and fewer parameters. The Decision Tree improved slightly but still did not perform as well as some of the other models.

Another way to look at the results is by comparing how close the top models are to each other. While SVM achieved the highest F1-score, the difference between SVM, XGBoost, and Random Forest is not very large. This suggests that several models are capable of learning the patterns in the dataset effectively. The slight advantage of SVM likely comes down to how it handles the transformed feature space rather than a major weakness in the other models.

Computational efficiency is another essential element. This is due to possible increased time costs and system burden. SVM provides strong performance, but it can be more expensive to train, especially with large datasets. In contrast, models like Random Forest and XGBoost may offer a better balance between performance and training time, which could make them more practical in real-world applications. Decision Tree took a full 46 minutes to execute in the original notebook. Choosing a model is not just about accuracy but also about how efficiently it can be applied in real life application.

The performance of Logistic Regression, a simple linear model, performed well. This suggests that the dataset has some level of linear separability, even though more complex models perform better overall. In other words, the relationships in the data are not purely nonlinear, which helps explain why multiple models perform well.

One important takeaway is that high accuracy does not always mean strong performance. Some models achieved high accuracy but had weaker precision or recall, especially for the minority class. The reason for this is that accuracy only measures overall correctness and does not account for class imbalance. A model can achieve high accuracy by predicting the majority class most of the time while missing important cases in the minority class. This is why metrics like precision, recall, and F1-score are more informative in this situation.

Precision and recall give more detail about how the model is performing. Precision looks at how many predicted positives are actually correct, while recall measures how many actual positives are correctly identified. F1-score combines both into a single value, which is why it was used as the main metric in this study. The SVM model shows a strong balance between precision and recall, which explains its high F1-score.

The confusion matrix for the SVM model gives a more detailed view of its performance. It correctly classified about 7,799 instances of Class 0 and 5,414 instances of Class 1. There were 907 false positives and 290 false negatives. These numbers show that the model performs well for both classes, especially when it comes to identifying coordinated behavior. The relatively low number of false negatives means the model is good at detecting the minority class, while the slightly higher number of false positives is an acceptable tradeoff.

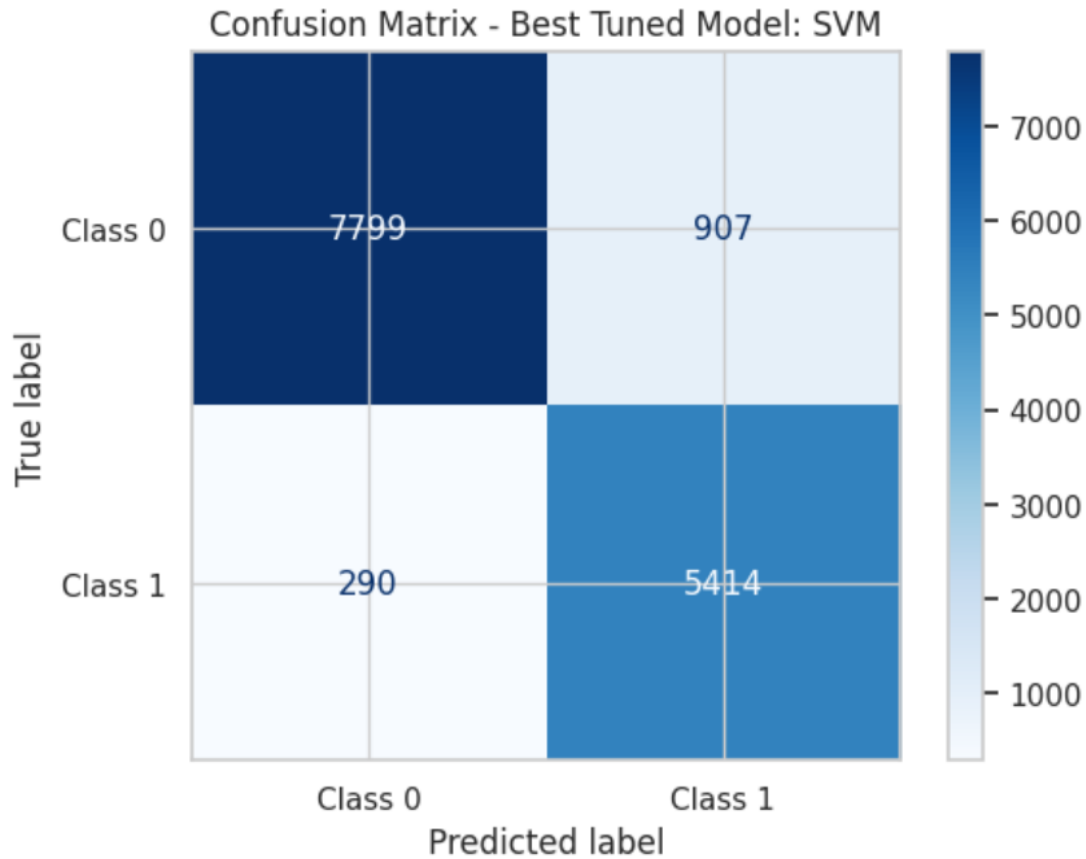


Figure 7 illustrates the classification performance of the SVM model across both classes.

The strong performance of ensemble models like Random Forest and XGBoost is also interesting. These models work by combining multiple decision trees, which helps improve predictions and reduce overfitting. Random Forest does this by averaging many trees trained on different subsets of the data, while XGBoost builds trees sequentially and focuses on correcting previous errors. Both approaches allow the models to capture complex patterns in the data. Even though they performed slightly below SVM, they still showed strong results.

Feature importance analysis using the Random Forest model provides additional insight into what drives predictions. The results show that velocity-related features, especially those related to movement, are among the most important. This suggests that how individuals move plays a larger role in determining whether behavior is coordinated than static features like position. This matches what we would expect from swarm systems, where coordination is often tied to consistent movement patterns.

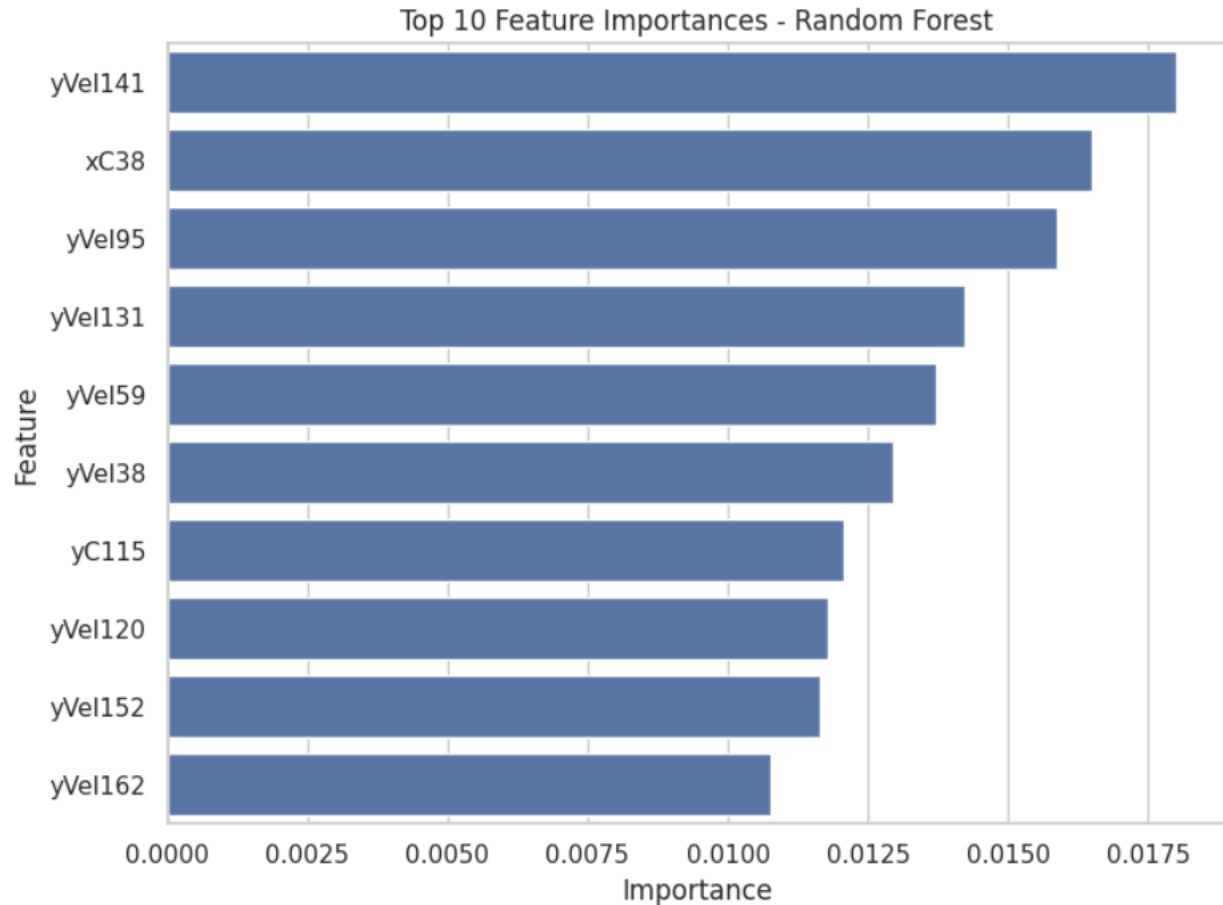


Figure 8 shows feature importance derived from the Random Forest model.

The weaker performance of the Decision Tree model can be explained by its tendency to overfit. Decision Trees create very specific rules based on the training data, which may not apply well to new data. This becomes more of a problem when the dataset has many features, since there are more possible splits. As a result, the model can end up learning noise instead of useful patterns. While tuning helped slightly, it was not enough to bring its performance in line with the other models.

Overall, the results show that machine learning models can successfully classify swarm behavior, with SVM performing the best for this dataset. The findings also show that preprocessing, dimensionality reduction, and hyperparameter tuning all play a role in improving performance. Just as importantly, they show the need to choose evaluation metrics carefully and match models to the structure of the data.

5. Model Behavior Analysis, Limitations, and Conclusion

This study shows that machine learning can be used effectively to classify swarm behavior, even when working with a high-dimensional dataset. By following a structured pipeline that included preprocessing, dimensionality reduction, model selection, and hyperparameter tuning, it was possible to achieve strong performance across multiple models. Among all the models tested, the SVM consistently performed the best, achieving an F1-score of about 0.9008. This suggests that choosing models that match the structure of the data is just as important as the modeling process itself.

One of the main reasons SVM performed so well is its ability to handle high-dimensional data. Instead of relying on all features equally, it focuses on finding the best boundary between classes by maximizing the margin. By relying on a smaller set of important data points, known as support vectors, the model is able to generalize well without being heavily influenced by noise or redundant features. Even after PCA reduced the feature space to 50 components, the data still contained enough complexity for SVM to perform effectively. This helps explain why it worked particularly well for this problem.

Principal Component Analysis also played a major role in making the dataset more manageable. Reducing the number of features from over 2,400 down to 50 components improved efficiency and helped reduce overfitting. At the same time, it kept about 55% of the total variance, which means most of the important patterns were still present. However, this comes with a tradeoff. Because PCA transforms the data into new components, it becomes harder to interpret what each feature represents. This makes it more difficult to understand exactly what is driving the model's predictions, which is something to keep in mind when working with similar datasets.

Hyperparameter tuning also had an impact on performance, especially for models like Random Forest and XGBoost. These models rely on several parameters, and adjusting them can improve how well they learn from the data. In this study, tuning led to noticeable improvements for these models. On the other hand, SVM did not change much after tuning, which suggests that its default settings were already a good fit. This demonstrates the idea that tuning is not always equally important for every model and should be considered based on the model and the dataset.

Using multiple evaluation metrics also helped provide a clearer picture of model performance. While accuracy is commonly used, it did not fully capture how well the models handled both classes. Some models achieved high accuracy but did not perform as well in terms of precision or recall. F1-score was used as the main metric, since it balances both. Looking at confusion matrices also made it easier to see how each model handled correct and incorrect predictions, especially for the minority class.

There are a few limitations to consider. One of the biggest limitations is the use of PCA. While it helps reduce dimensionality and improve efficiency, it also makes the model harder to interpret. Since the principal components are combinations of the original features, it is not always clear which specific features are most important. This can be a drawback in situations where understanding the model's decisions is important.

Another limitation is that the dataset is based on simulations. While simulation allows for large amounts of controlled data, it may not fully represent real-world swarm behavior. Real systems can include more noise, environmental variation, and unpredictable behavior. Model performance might change when applied to real-world data. This makes it important to test these models on real datasets in the future to see how well they generalize.

The lack of temporal modeling is another limitation. Swarm behavior changes over time, but in this dataset, each observation is treated independently. This means the models do not capture how behavior evolves. Including time-based features or using models designed for sequential data could provide a better understanding of swarm dynamics and potentially improve performance.

There are also limitations related to hyperparameter tuning. While GridSearchCV and RandomizedSearchCV were used, the search space was limited to specific parameter ranges. Expanding this search or using more advanced methods like Bayesian optimization could lead to better results. However, this would also increase computational cost, which is already fairly high for this type of dataset.

Another thing to consider is how dimensionality reduction might affect performance. While PCA improves efficiency, it may remove smaller patterns that could still be useful for classification. The choice to use 50 components worked well here, but exploring different numbers of components could provide more insight into how feature reduction impacts model performance.

There are several ways this work could be extended. One option is to use feature selection instead of PCA. Feature selection keeps the original features but removes the less useful ones, which can make the model easier to interpret. Another way is to explore deep learning models, which may be able to capture more complex patterns in the data.

Incorporating time-based modeling is another area. Models like recurrent neural networks or LSTM networks are designed to work with sequential data and could help capture how swarm behavior changes over time. This could lead to better predictions and a deeper understanding of the system.

Using real-world data would also be an important next step. Real datasets would provide a more realistic test of model performance and help identify any gaps between simulation and real behavior. This would make the findings more applicable in practical settings.

Exploring more advanced tuning methods could improve performance. Alternative techniques, such as Bayesian optimization or evolutionary algorithms, can search for parameter spaces more efficiently. These alternative techniques have the potential to result in better configurations than standard methods. This is especially useful for more complex models like XGBoost.

This study shows that machine learning models can be used to classify swarm behavior in high-dimensional data, with the SVM performing the best overall. It also demonstrates how preprocessing, dimensionality reduction, and evaluation of choices all contribute to model performance. While there are limitations, the results provide a strong foundation for developing advanced methods.

References

- Bonabeau, E., Dorigo, M., & Theraulaz, G. (1999). *Swarm intelligence: From natural to artificial systems*. Oxford University Press.
- Brambilla, M., Ferrante, E., Birattari, M., & Dorigo, M. (2013). Swarm robotics: A review. *Swarm Intelligence*, 7(1), 1–41. <https://doi.org/10.1007/s11721-012-0075-2>
- Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861–874.
- Jolliffe, I. T. (2002). *Principal component analysis* (2nd ed.). Springer.
- Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. *Proceedings of ICNN'95*, 1942–1948.
- Reynolds, C. W. (1987). Flocks, herds, and schools: A distributed behavioral model. *SIGGRAPH Computer Graphics*, 21(4), 25–34.